

Subquery

Baza danych: PostgreSQL

Subquery – one row

	A-Z first_name ▼	A-Z last_name ▼	123 department_id ▼
1	Matthew	Weiss	50
2	Adam	Fripp	50
3	Payam	Kaufling	50
4	Shanta	Vollman	50
5	Kevin	Mourgos	50
6	Julia	Nayer	50
7	Irene	Mikkilineni	50
8	James	Landry	50

```
select
first_name,
last_name,
department_id
from employees
where
department_id =
(select department_id from departments where department_name
= 'Shipping')
```

Subquery –
one row -
error



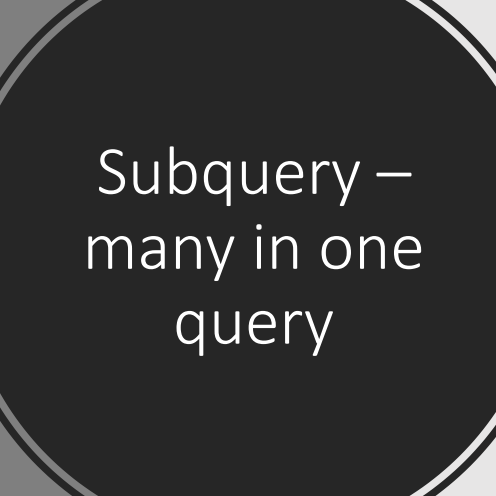
SQL Error [21000]: BŁĄD: ponad
jeden wiersz zwrócony przez
podzapytanie użyte jako
wyrażenie

```
select
    first_name,
    last_name,
    department_id
from employees
where
    department_id =
    (select department_id from departments
     where department_name in ('Shipping', 'Administration'))
```

Subquery –
more than
one row

	A-Z first_name ▼	A-Z last_name ▼	123 department_id ▼
1	Matthew	Weiss	50
2	Adam	Fripp	50
3	Payam	Kaufling	50
4	Shanta	Vollman	50
5	Kevin	Mourgos	50
6	Julia	Nayer	50
7	Irene	Mikkilineni	50

```
select
    first_name,
    last_name,
    department_id
from employees
where
    department_id IN
        (select department_id from departments
         where department_name in
             ('Shipping', 'Administration'))
```



Subquery –
many in one
query

	A-Z first_name	A-Z last_name	123 department_id
1	David	Williams	60
2	Valli	Jackson	60
3	Diana	Nguyen	60
4	Nandita	Sarchand	50
5	Alexis	Bull	50
6	Sarah	Bell	50
7	Jennifer	Whalen	10

select

```
first_name,  
last_name,  
department_id
```

from employees

```
where salary between (select min_salary from jobs
                      where job_title = 'Programmer')
and (select max_salary from jobs
     where job_title = 'Stock Clerk')
```

Subquery – in having

	123 department_id ▼	123 sum ▼
1	70 	10,000
2	[NULL]	7,000
3	10	4,400
4	40	6,500

```
select
    department_id,
    sum(salary)
from employees
group by department_id
having sum(salary) < (select sum(salary) from employees
                      where department_id = 20)
```


Subquery –
operator
ANY

	A-Z first_name	A-Z last_name	123 salary	123 department_id
1	Steven	King	24,000	90
2	Neena	Yang	17,000	90
3	Lex	Garcia	17,000	90
4	Alexander	James	9,000	60
5	Nancy	Gruenberg	12,008	100
6	Daniel	Faviet	9,000	100
7	John	Chen	8,200	100

```
select
first_name,
last_name,
salary,
department_id
from employees
where salary > ANY (select salary from employees
                     where department_id = 20)
```


Subquery –
operator ALL

	A-Z first_name	A-Z last_name	123 salary	123 department_id
1	Steven	King	24,000	90
2	Neena	Yang	17,000	90
3	Lex	Garcia	17,000	90
4	John	Singh	14,000	80
5	Karen	Partners	13,500	80

```
select
    first_name,
    last_name,
    salary,
    department_id
from employees
where salary > ALL (select salary from employees
                    where department_id = 20)
```


Subquery – correlation

	A-Z first_name ▼	A-Z last_name ▼	123 salary ▼	123 department_id ▼
1	Steven	King	24,000	90
2	Alexander	James	9,000	60
3	Bruce	Miller	6,000	60
4	Nancy	Gruenberg	12,008	100
5	Daniel	Faviet	9,000	100
6	Den	Li	11,000	30
7	Matthew	Weiss	8,000	50

```
select
first_name,
last_name,
salary,
department_id
from employees e
where salary > (select avg(salary) from employees
                where department_id = e.department_id)
```

Subquery – correlation - EXISTS

	A-Z department_name ▼
1	Administration
2	Marketing
3	Purchasing
4	Human Resources
5	Shipping
6	IT
7	Public Relations
8	Sales
9	Executive
10	Finance
11	Accounting

```
select  
department_name  
from departments d  
where  
    EXISTS (select 1 from employees  
              where department_id = d.department_id )
```

Subquery – correlation – NOT EXISTS

	A-Z department_name ▼
1	Operations
2	Corporate Tax
3	Benefits
4	Payroll
5	Recruiting
6	Treasury
7	Retail Sales
8	IT Support

```
select  
department_name  
from departments d  
where  
    NOT EXISTS (select 1 from employees  
                where department_id = d.department_id )
```

Problem



SQL Error [42703]: BŁĄD:
kolumna "full_salary" nie
istnieje

```
select  
  first_name,  
  last_name,  
  salary + (salary * coalesce(commission_pct,0)) full_salary  
from employees e  
where full_salary > 5000
```

Problem –
resolve by
subquery
- table
derived

	A-Z first_name ▼	A-Z last_name ▼	123 full_salary ▼
1	Steven	King	24,000
2	Neena	Yang	17,000
3	Lex	Garcia	17,000
4	Alexander	James	9,000
5	Bruce	Miller	6,000
6	Nancy	Gruenberg	12,008
7	Daniel	Faviet	9,000

```
select
    * from (
        select
            first_name,
            last_name,
            salary + (salary *
                coalesce(commission_pct,0)) full_salary
        from employees e ) t1
where t1.full_salary > 5000
```

Subquery in select - derived column

	A-Z first_name ▼	A-Z last_name ▼	A-Z dep_name ▼	
1	Steven	King	Executive	
2	Neena	Yang	Executive	
3	Lex	Garcia	Executive	
4	Alexander	James	IT	
5	Bruce	Miller	IT	

```
select
first_name,
last_name,
(select department_name from departments where
    department_id = e.department_id) dep_name
from employees e
```

Subquery many columns

	A-Z last_name ▼	123 salary ▼
1	King	24,000
2	James	9,000
3	Gruenberg	12,008
4	Li	11,000
5	Fripp	8,200
6	Singh	14,000
7	Whalen	4,400
8	Martinez	13,000
9	Jacobs	6,500
10	Brown	10,000
11	Higgins	12,008

```
select  
e.last_name,  
e.salary  
from employees e  
where  
  (department_id, salary) in  
    (select department_id, max(salary) from employees  
      group by department_id)
```

CTE

	A-Z last_name ▼	123 salary ▼
1	King	24,000
2	Yang	17,000
3	Garcia	17,000
4	Gruenberg	12,008
5	Li	11,000
6	Singh	14,000
7	Partners	13,500
8	Errazuriz	12,000

```
WITH high_earn AS (  
    SELECT last_name, salary  
    FROM employees  
    WHERE salary > 10000)  
SELECT *  
FROM high_earn;
```


CTE

	○ A-Z opis ▼	123 wartosci ▼
1	Ilość:	107
2	Suma wypłat:	691,416

```
WITH dane AS (  
    SELECT * FROM employees  
)  
SELECT 'Ilość: ' opis, COUNT(*) wartosci FROM employees  
UNION ALL  
SELECT 'Suma wypłat: ', SUM(salary) FROM employees;
```

CTE - hierarchy

```
WITH RECURSIVE hierarchia AS (
    -- Kotwica: pracownicy bez managera (top-level)
    SELECT
        employee_id,
        last_name,
        manager_id,
        1 AS poziom,
        last_name::TEXT AS sciezka
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    -- Rekurencja: dołączamy podwładnych
    SELECT
        e.employee_id,
        e.last_name,
        e.manager_id,
        h.poziom + 1,
        h.sciezka || ' > ' || e.last_name
    FROM employees e
    JOIN hierarchia h ON e.manager_id = h.employee_id)

SELECT
    *
FROM hierarchia
ORDER BY sciezka;
```

	123 employee_id	AZ last_name	123 manager_id	123 poziom	AZ sciezka
1	100	King	[NULL]	1	King
2	148	Cambrault	100	2	King > Cambrault
3	172	Bates	148	3	King > Cambrault > Bates
4	169	Bloom	148	3	King > Cambrault > Bloom
5	170	Fox	148	3	King > Cambrault > Fox
6	173	Kumar	148	3	King > Cambrault > Kumar



CTE - materialized

MATERIALIZED

- PostgreSQL wykonuje zapytanie wewnątrz CTE **tylko raz**.
- Wynik jest zapisywany w pamięci (lub na dysku, jeśli jest duży) jako tymczasowa tabela.
- **Zaleta:** Jeśli używasz tego samego CTE 5 razy w jednym zapytaniu, baza nie przelicza go 5 razy.
- **Wada:** Optymalizator "nie widzi" co jest w środku. Jeśli w głównym zapytaniu dasz `WHERE id = 10`, baza i tak najpierw pobierze **wszystkie** dane do CTE, a dopiero potem przefiltruje wynik.

NOT MATERIALIZED (domyślne dla prostych CTE)

- PostgreSQL "wkleja" kod CTE bezpośrednio do głównego zapytania (tzw. *inlining*).
- **Zaleta:** Optymalizator może "przepchnąć" filtry z głównego zapytania do środka CTE. Jeśli masz `WHERE id = 10` na zewnątrz, baza użyje indeksu na `id` wewnątrz CTE.
- **Wada:** Jeśli używasz CTE wielokrotnie, może zostać wykonane kilka razy, co obciąży procesor.

CTE - materialized

	A-Z opis ▼	123 wartosci ▼
1	Ilość:	107
2	Suma wypłat:	691,416

```
-- Wymuszenie materializacji (zapisanie wyniku do tymczasowej struktury)
WITH dane AS MATERIALIZED (
    SELECT
        e.last_name,
        d.department_id,
        d.department_name
    FROM employees e join departments d on e.department_id = d.department_id
)
SELECT * FROM dane WHERE department_id < 100;
```



Plan dla zmaterializowanego CTE (CTE Scan)

A-Z QUERY PLAN
CTE Scan on dane (cost=13.07..15.48 rows=36 width=158)
Filter: (department_id < '100'::numeric)
CTE dane
-> Nested Loop (cost=0.16..13.07 rows=107 width=96)
-> Seq Scan on employees e (cost=0.00..3.07 rows=107 width=11)
-> Memoize (cost=0.16..0.63 rows=1 width=90)
Cache Key: e.department_id
Cache Mode: logical
-> Index Scan using dept_id_pk on departments d (cost=0.15..0.62 rows=1 width=90)
Index Cond: (department_id = e.department_id)

Plan bez materializacji (Scan tabel)

A-Z QUERY PLAN
Nested Loop (cost=0.16..13.10 rows=36 width=96)
-> Seq Scan on employees e (cost=0.00..3.07 rows=107 width=11)
-> Memoize (cost=0.16..0.63 rows=1 width=90)
Cache Key: e.department_id
Cache Mode: logical
-> Index Scan using dept_id_pk on departments d (cost=0.15..0.62 rows=1 width=90)
Index Cond: ((department_id = e.department_id) AND (department_id < '100'::numeric))



CTE - materialized

Scenariusz	Rekomendacja
CTE jest używane tylko raz w zapytaniu	NOT MATERIALIZED (zazwyczaj domyślne)
CTE jest bardzo kosztowne i używane wielokrotnie	MATERIALIZED
Chcesz użyć indeksu z tabeli wewnątrz CTE, filtrując na zewnątrz	NOT MATERIALIZED
Chcesz stworzyć "punkt stopu" dla optymalizatora (Optimization Fence)	MATERIALIZED

Optimization Fence to sytuacja, w której optymalizator zapytania (Planner) nie jest w stanie „zajrzeć” do środka fragmentu kodu SQL, aby zoptymalizować go wspólnie z resztą zapytania.



CTE - materialized

Optimization Fence to sytuacja, w której optymalizator zapytania (Planner) nie jest w stanie „zajrzeć” do środka fragmentu kodu SQL, aby zoptymalizować go wspólnie z resztą zapytania.

Jeśli CTE jest **zmaterializowane** (czyli działa jako Optimization Fence):

- Baza najpierw wykonuje **całe** zapytanie wewnątrz CTE (skanuje 10 mln wierszy).
- Zapisuje wynik do tymczasowej tabeli w pamięci.
- Dopiero potem przeszukuje tę tymczasową tabelę, aby znaleźć id = 500.
- **Efekt:** Bardzo wolne działanie, mimo że na kolumnie id w tabeli sprzedaz może być indeks. Filtr WHERE id = 500 nie „przeszedł” przez płot do środka CTE.

Główną wadą jest **brak propagacji filtrów** (Predicate Pushdown). Optymalizator nie może użyć indeksów z tabel źródłowych, jeśli warunek filtrujący znajduje się poza zmaterializowanym CTE.



CTE - materialized

Kiedy to jest przydatne?

Choć brzmi to jak ograniczenie, Optimization Fence bywa potężnym narzędziem w rękach dewelopera:

- **Wymuszenie kolejności wykonywania:** Jeśli wiesz, że optymalizator wybiera zły plan (np. najpierw łączy ogromne tabele, a potem filtruje), możesz zamknąć filtrowanie w zmaterializowanym CTE, zmuszając bazę do wykonania go najpierw.
- **Zapobieganie błędom (Side Effects):** Jeśli masz funkcję, która jest kosztowna lub ma skutki uboczne, możesz ją „odizolować” w CTE, aby mieć pewność, że wykona się dokładnie tyle razy, ile zakładasz.
- **Stabilizacja planu:** W bardzo skomplikowanych zapytaniach optymalizator czasem „gubi się” w statystykach. Postawienie „fence” upraszcza problem dla planneru, dzieląc jedno wielkie zadanie na dwa mniejsze, przewidywalne etapy.